



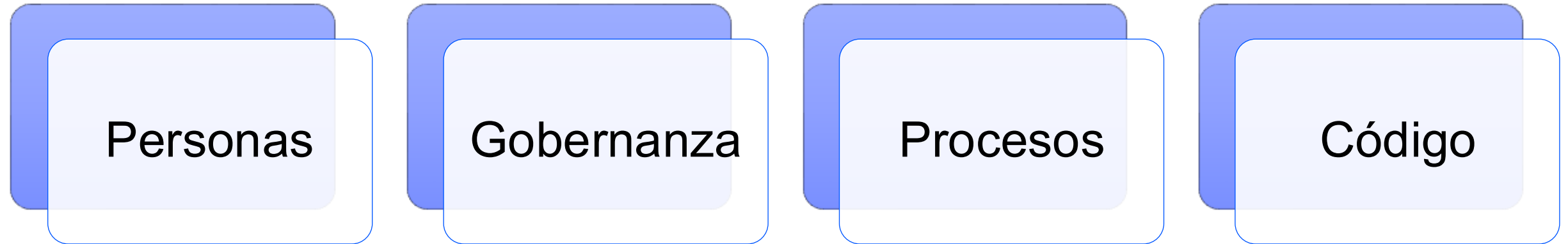
Gobernanza y Desarrollo de CPython

El Rol de los Core Developers

Diego Russo
Universidad Nacional de Salta · 12 de agosto de 2026

Python parece un lenguaje

CPython es un proyecto de ingeniería



Hoy vamos a descubrir cómo se construye Python

¿Por qué mirar detrás de escena?

Porque CPython es uno de los mejores ejemplos de ingeniería de software colaborativa



Arquitectura

Colaboración

Calidad

Mantenimiento

¿Quién soy?

Diego Russo



diego@python.org



CPython Core Developer

- JIT de CPython
- Rendimiento del runtime
- Componentes internos del compilador y del intérprete

Principal Software Engineer

- Lidero el trabajo de CPython en Arm
- Desarrollo del ecosistema Python sobre Arm

Comunidad y conferencias

- Organizador y expositor en EuroPython
- Charla magistral en PyCon Italia
- Expositor en PyCon UK

¿En qué trabajo dentro de CPython?

El trabajo combina código, decisiones y comunidad

Runtime

- Desarrollo y mantenimiento del intérprete

Rendimiento

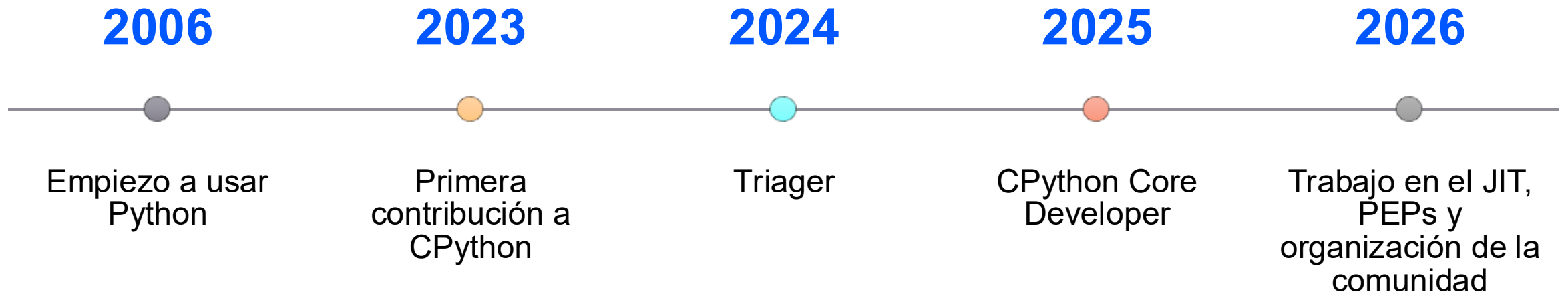
- JIT, optimizaciones y plataformas

Comunidad

- PEPs, revisiones, conferencias y Core Developer Sprints

Mi recorrido

Todo empezó con un primer aporte



¿Por qué les cuento todo esto?

Porque hace unos años yo estaba exactamente del otro lado

Usaba Python

No conocía el proyecto

Pensaba que contribuir era imposible

gh-110190: Fix ctypes structs with array on Arm #1

Merged [ambv](#) merged 6 commits into [python:main](#) from [diegorusso:main](#) on Dec 5, 2023

Conversation **30** Commits **6** Checks **0** Files changed **4**



diegorusso commented on Dec 1, 2023 • edited

Member

Set MAX_STRUCT_SIZE to 32 in stgdict.c when on Arm platforms.
This because on Arm platforms structs with at most 4 elements of any floating point type values can be passed through registers. If the type is double the maximum size of the struct is 32 bytes.
By Arm platforms it is meant 32 and 64 bit.
On x86-64 Linux, it's maximum 16 bytes hence we need to differentiate.

- Issue: [ctypes array cannot be returned by value on ARM64 #110190](#)



Python

- El lenguaje



CPython

- La implementación de referencia

CPython es mucho más que un intérprete

Runtime

Biblioteca
estándar

Compilador

Tests

Documentación

Herramientas

Releases

Seguridad

Un proyecto usado por millones de personas

Cada cambio tiene consecuencias

35+ años de desarrollo

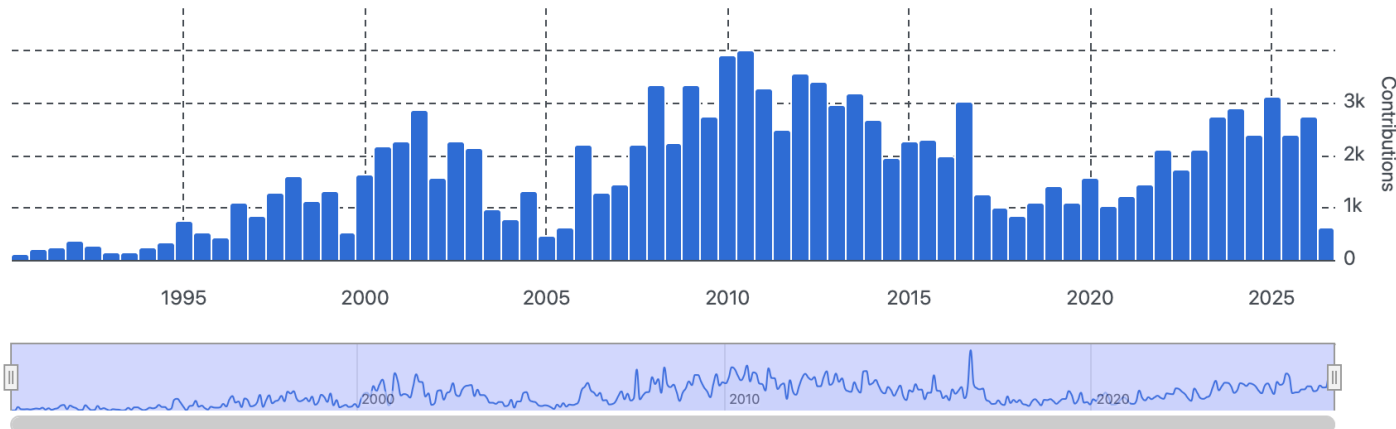
Millones de usuarios

Miles de contribuciones

Decenas de plataformas y arquitecturas

Commits over time

Weekly from 4 Aug 1990 to 1 Aug 2026



Las personas detrás de Python

Una comunidad que construye, mantiene y hace evolucionar el lenguaje

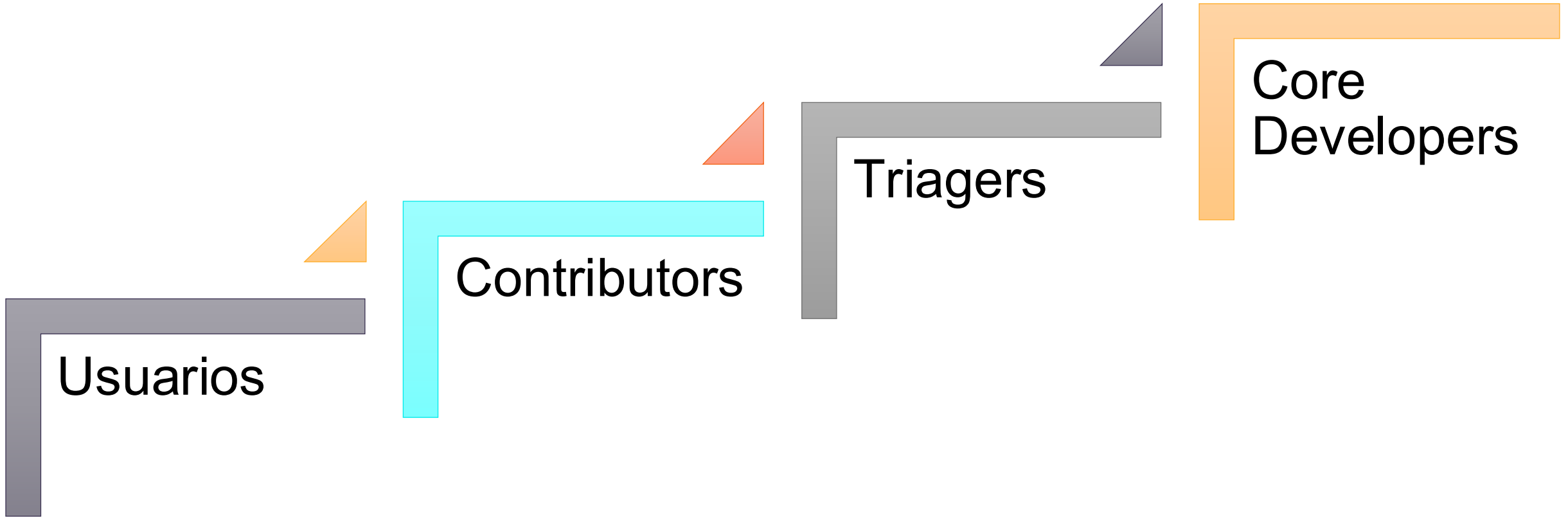
Python lo construyen personas

Detrás de cada línea de código hay una comunidad de personas



¿Quiénes participan?

Cada rol implica un nivel diferente de responsabilidad



¿Qué hace un Core Developer?

La mayor responsabilidad de un Core Developer es mantener Python

Revisar

- Contribuciones de otras personas

Integrar

- Aceptar cambios al repositorio

Mantener

- Corregir regresiones y cuidar el código existente

Mentorear

- Acompañar a nuevos colaboradores

Representar al proyecto

- Participar en discusiones técnicas y en la comunidad

¿Qué significa ser Core Developer?

Los beneficios existen para facilitar el trabajo



Confianza

La comunidad confía en tu criterio técnico



Responsabilidad

Podés integrar cambios y ayudar a mantener el proyecto



Comunidad

Participás en los Core Developer Sprints, conferencias y discusiones técnicas



Herramientas

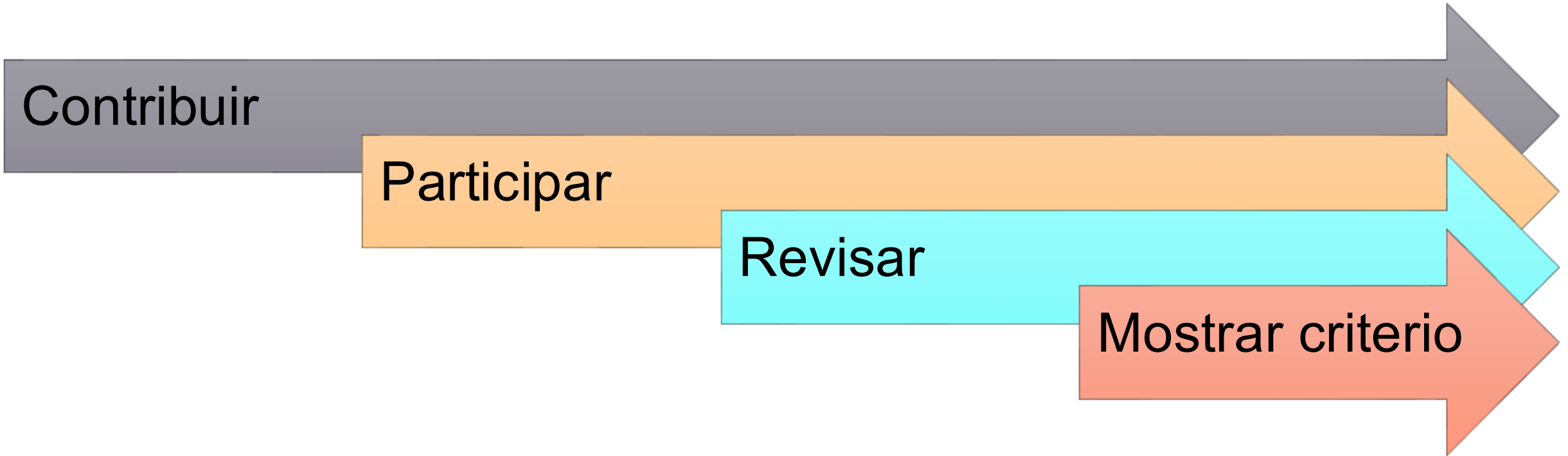
Podés patrocinar PEPs, disponer de permisos de merge y colaborar más directamente con el resto del equipo

Fuente: Mariatta Wijaya, “Perks of being a Python Core Developer”

<https://mariatta.ca/posts/perks-of-python-core/>

¿Cómo se llega a ser Core Developer?

Proceso basado en contribuciones sostenidas y en la confianza de la comunidad



Ser constante a lo largo del tiempo

Mi promoción a Core Developer

<https://devguide.python.org/core-team/join-team/>

Vote to promote Diego Russo

■ Committers ■ promotion



Łukasz Langa  ambv CPython core developer

2 May

Hi,

Pablo and I would like to propose promoting **Diego Russo** ([diegorusso](#) 48 on GitHub) as a core developer.

Promote Diego Russo

100%



Do not promote

0%

43

voters

 Display tally

No hace falta ser especialista

Hay muchas maneras de aportar valor al proyecto

Tests

Documentación

Biblioteca
estándar

Packaging

Herramientas

CI

Rendimiento

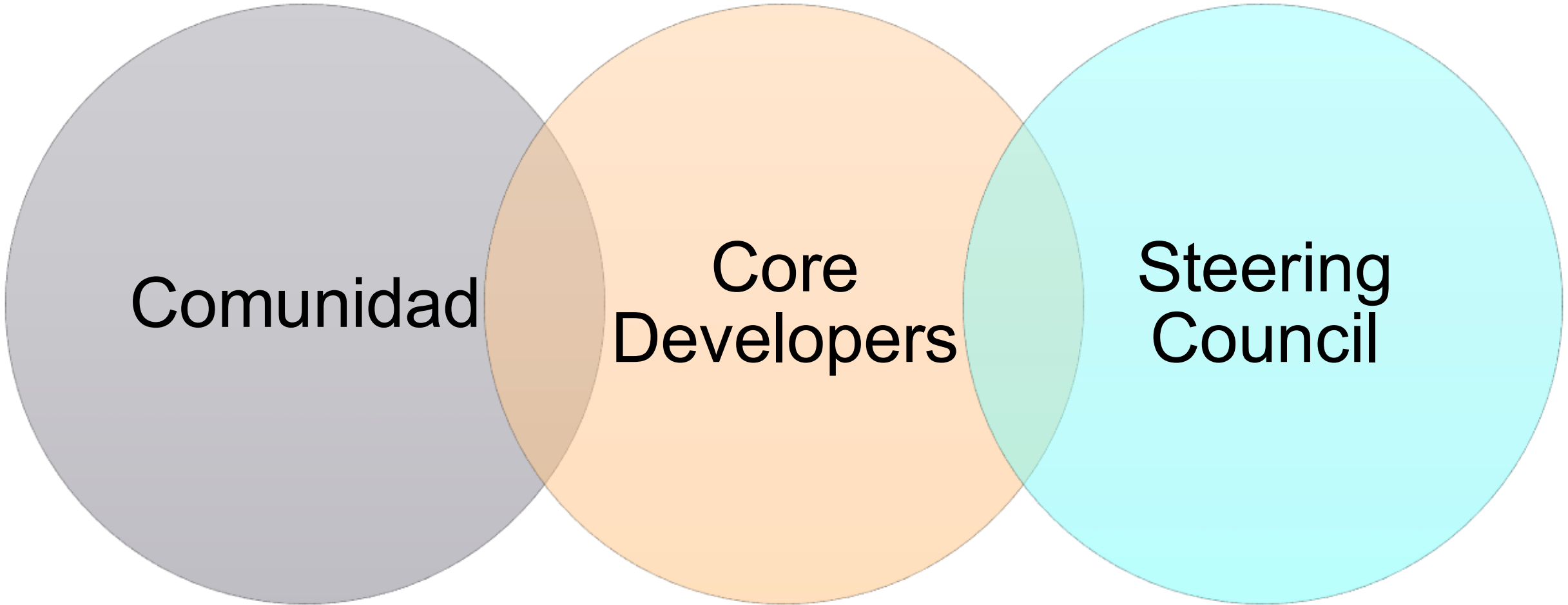
Revisiones

Cómo se toman las decisiones

La gobernanza permite que miles de personas trabajen sobre un mismo proyecto

¿Quién toma las decisiones?

Las decisiones importantes no dependen de una sola persona



¿Dónde ocurre la comunicación?



GitHub



discuss.python.org



Discord



Conferencias



Sprints

Las conversaciones en persona complementan lo que queda escrito

Del BDFL al Steering Council

PEP 13

1991–2018

Guido van Rossum
BDFL (Benevolent Dictator For Life)

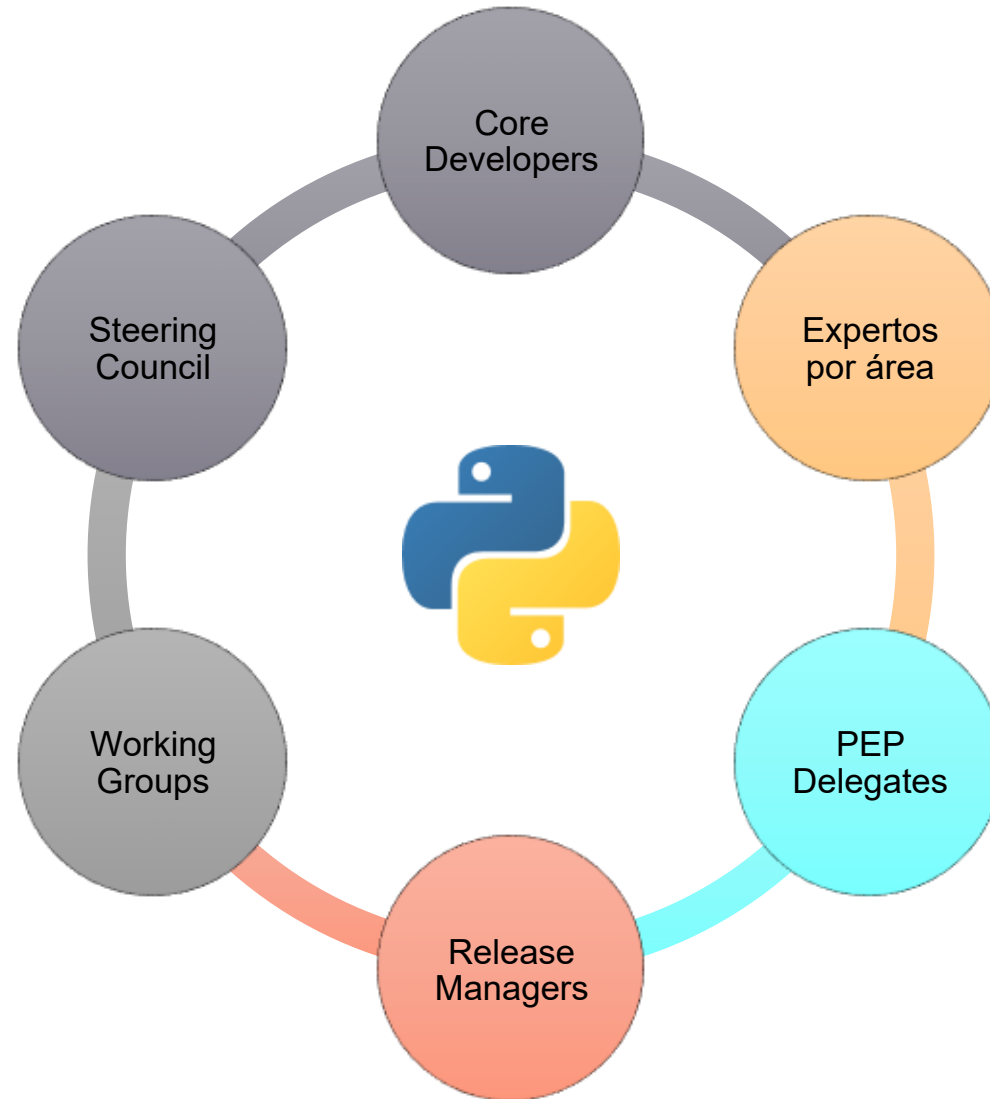


2019 hasta hoy

Steering Council
5 miembros elegidos



La autoridad está distribuida

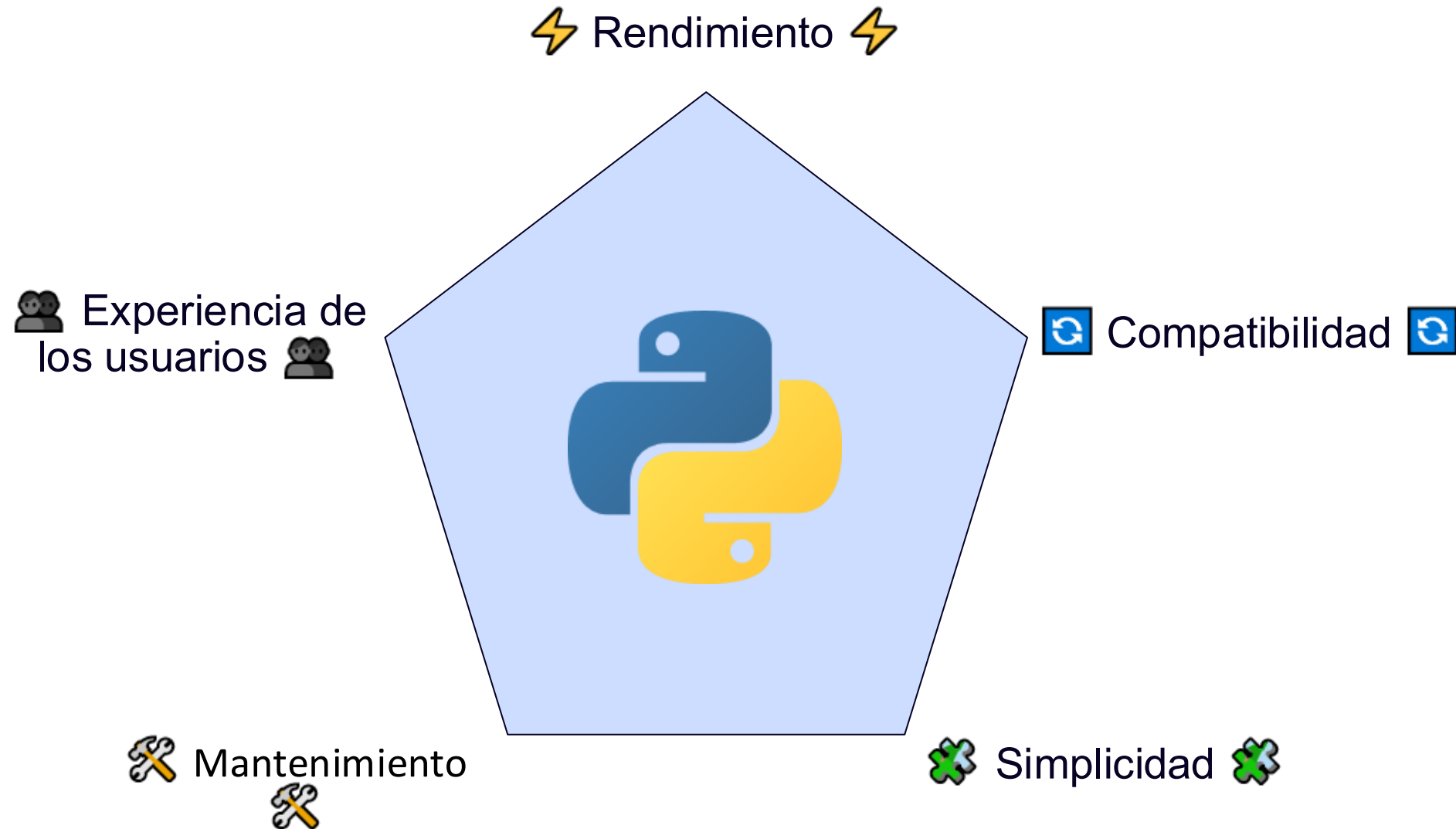


Cómo evoluciona Python

Desde una idea hasta un cambio en el código

Cambiar Python significa tomar decisiones

La ingeniería consiste en encontrar el mejor equilibrio



Todo cambio empieza con una hipótesis

La evidencia es más importante que la opinión

Bug

“Esto no funciona correctamente”

- Necesitamos reproducirlo

Nueva funcionalidad

“Esto resolvería un problema”

- Necesitamos justificarla

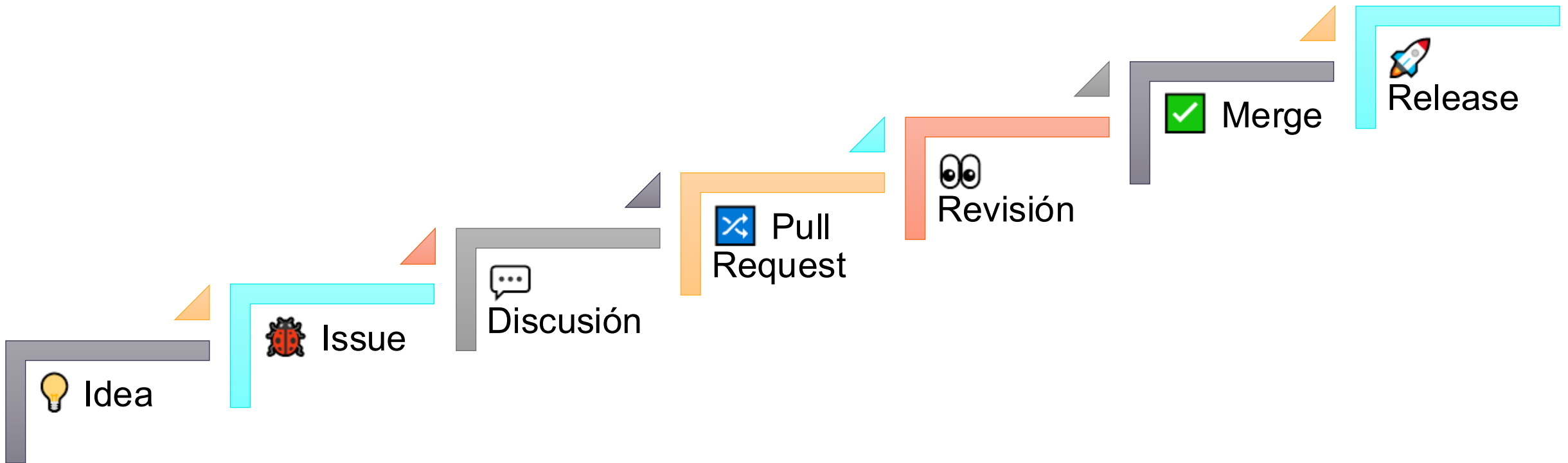
Rendimiento

“Esto hace Python más rápido”

- Necesitamos medirlo

¿Cómo llega un cambio a Python?

Todo cambio sigue un proceso abierto y colaborativo



CI revisa máquinas. Las personas revisan ingeniería

Que los tests pasen no significa que el cambio esté listo

CI



☐ Compila

☐ Ejecuta tests

☐ Prueba en distintas plataformas

☐ Usa sanitizers

Revisión



☐ Evalúa diseño

☐ Compatibilidad

☐ API

☐ Mantenimiento

La revisión mejora el código



pablogsal reviewed on May 2

[View reviewed changes](#)

✓ Include/internal/pycore_jit_unwind.h **Outdated**

```
10      10
11      11      #if defined(_Py_JIT) && defined(__linux__) && defined(__ELF__)
12      12      #   define PY_HAVE_JIT_GDB_UNWIND
13      + #   if defined(HAVE_EXECINFO_H) && defined(HAVE_BACKTRACE)
```



pablogsal on May 2

Member



This needs a separate configure link probe. `HAVE_BACKTRACE` only tells us that `backtrace()` is available, but this code also calls libgcc-specific symbols: `__register_frame` and `__deregister_frame`.

For example, Linux builds with `clang -rtlib=compiler-rt` can have `execinfo.h` / `backtrace()` while those frame registration symbols are not linkable. In that case thismacro gets enabled and the build fails later at link time.

Please add a configure check for `__register_frame` `__deregister_frame` and gate `PY_HAVE_JIT_GNU_BACKTRACE_UNWIND` on that result.



1



pablogsal on May 2 • edited ▾

Member

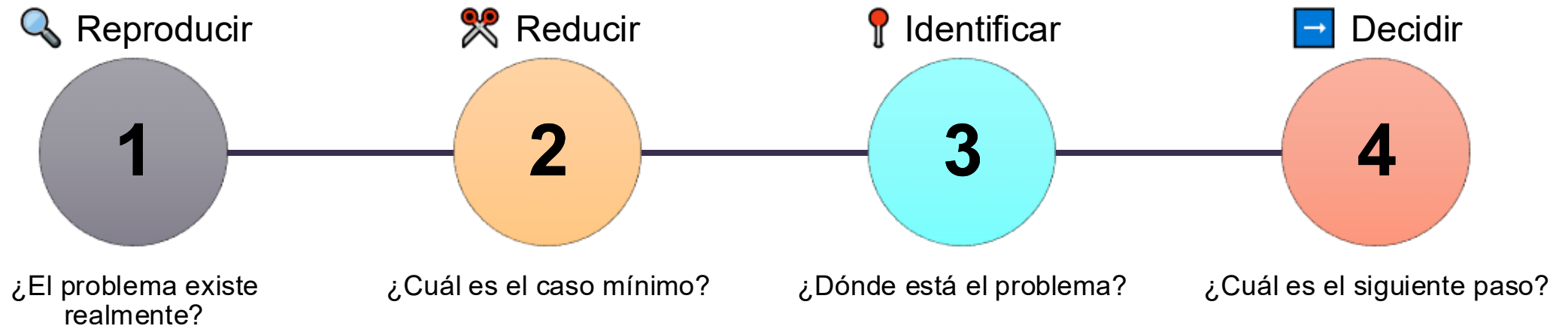


For example this can break in:

- Alpine / musl + libexecinfo (no libgcc_s `__register_frame`)
- clang + compiler-rt without libgcc_s
- LLVM libunwind builds (different FDE table format)

Antes de escribir una línea de código

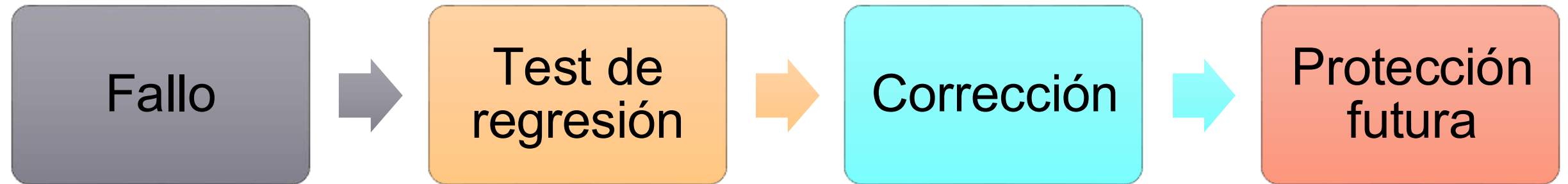
Entender el problema suele llevar más tiempo que escribir la solución



Este proceso se llama “trriage”

Los tests son memoria ejecutable

Un test conserva la historia de un fallo



El merge no es el final

El código pasa a formar parte de Python durante muchos años

 Backports 

¿Qué versiones reciben el cambio?

 Regresiones 

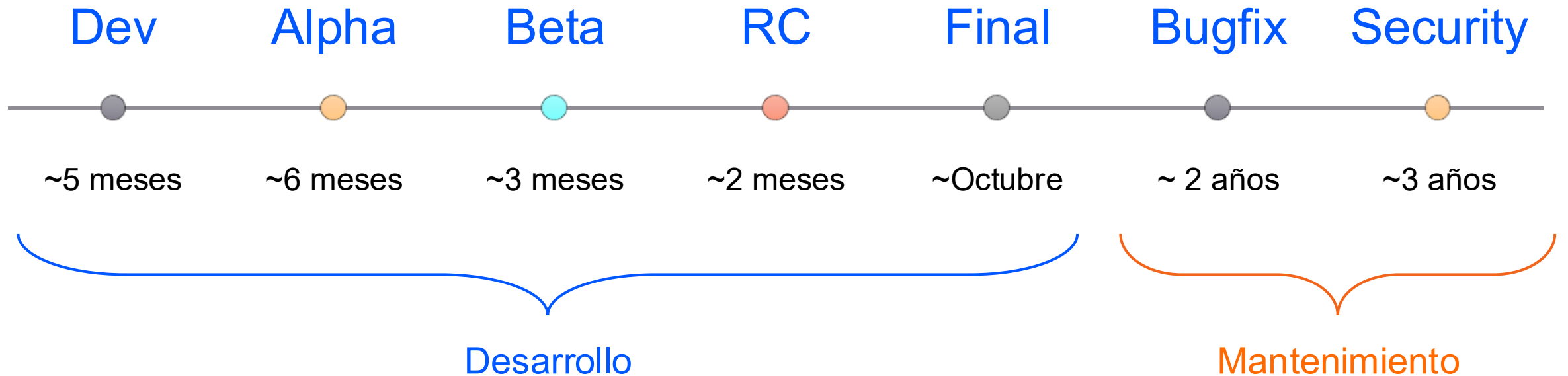
¿El cambio rompió algo?

 Mantenimiento 

¿Quién se hace responsable de ese código?

Una release convierte código en promesas

Una versión genera aproximadamente cinco años de mantenimiento



Diseñando el lenguaje

Los cambios importantes necesitan
discusión, evidencia y consenso

PEP

Python Enhancement Proposal
Problema → Propuesta → Consecuencias

Tipos de PEPs

Cada tipo de PEP responde a una necesidad diferente

Estándares (Standards Track)

Proceso (Process)

Informativos (Informational)

¿Quién decide un PEP?

No se decide por popularidad, sino por argumentos, evidencia y sostenibilidad

1

Comunidad

- Discute y aporta evidencia

2

Autor y revisores

- Refinan la propuesta

3

PEP Delegate o Steering Council

- Toma la decisión final

También podés contribuir a un PEP

Contribuir también es hacer mejores las ideas de otras personas

 Leer la propuesta

 Participar en la discusión

 Revisar el contenido técnico

 Mejorar la redacción

 Probar la implementación

-
-   **PEP 824: None-coalescing operators** ✓ new-pep
#4974 opened on May 18 by JelleZijlstra Member  30 tasks
-
- ☐  **PEP 824: None-coalescing operators** ✓ new-pep
#4799 opened on Jan 31 by cdce8p Contributor  24 of 30 tasks
-
- ☐  **PEP 823: None-aware access operators** ✓ new-pep
#4798 opened on Jan 31 by cdce8p Contributor  24 of 30 tasks
-
- ☐  **PEP 805: Safe Parallel Python** ✓ new-pep
#4579 opened on Sep 8, 2025 by markshannon Member
-
- ☐  **PEP 796: Relative Virtual Environments** ✗ new-pep
#4476 opened on Jun 30, 2025 by rickeylev First-time contributor • Changes
-
- ☐  **PEP 795: Deep Immutability in Python** ✓ new-pep
#4468 opened on Jun 19, 2025 by TobiasWrigstad First-time contributor 
-
- ☐  **PEP 717: Delegated Repository Authentication** ✓ new-pep

Un vistazo por dentro de CPython

La arquitectura que ejecuta nuestros
programas

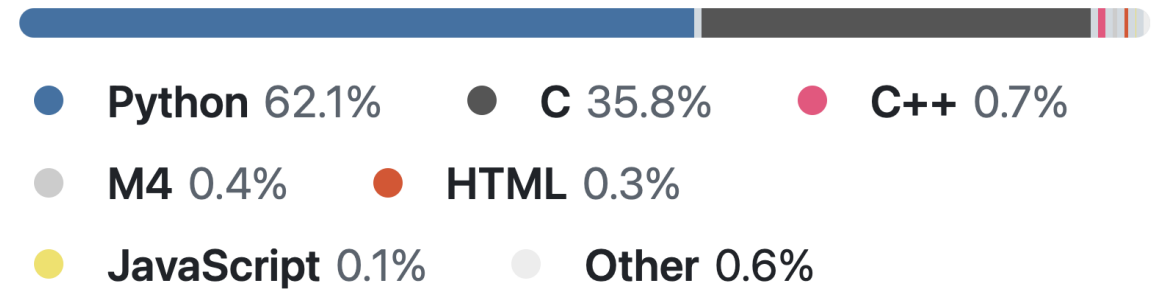
arm

El repositorio de CPython

Cada directorio tiene una responsabilidad bien definida

Python/	Runtime e intérprete
Objects/	Tipos incorporados
Include/	Cabeceras e interfaces
Lib/	Biblioteca estándar
Lib/test/	Tests de CPython
Modules/	Módulos escritos en C
Parser/	Parser de Python
Grammar/	Definición formal del lenguaje
Tools/	Herramientas de desarrollo
Doc/	Documentación

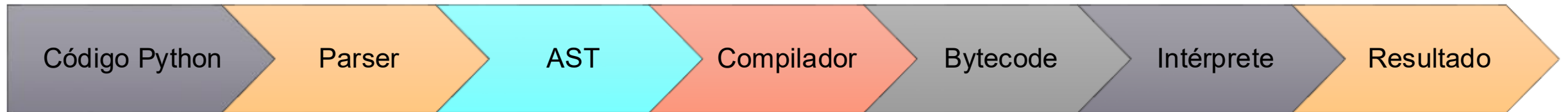
Languages



Fuente: repositorio [python/cpython](#) en GitHub, consultado en agosto de 2026

Un viaje muy rápido por CPython

CPython transforma el programa en varias etapas antes de ejecutarlo



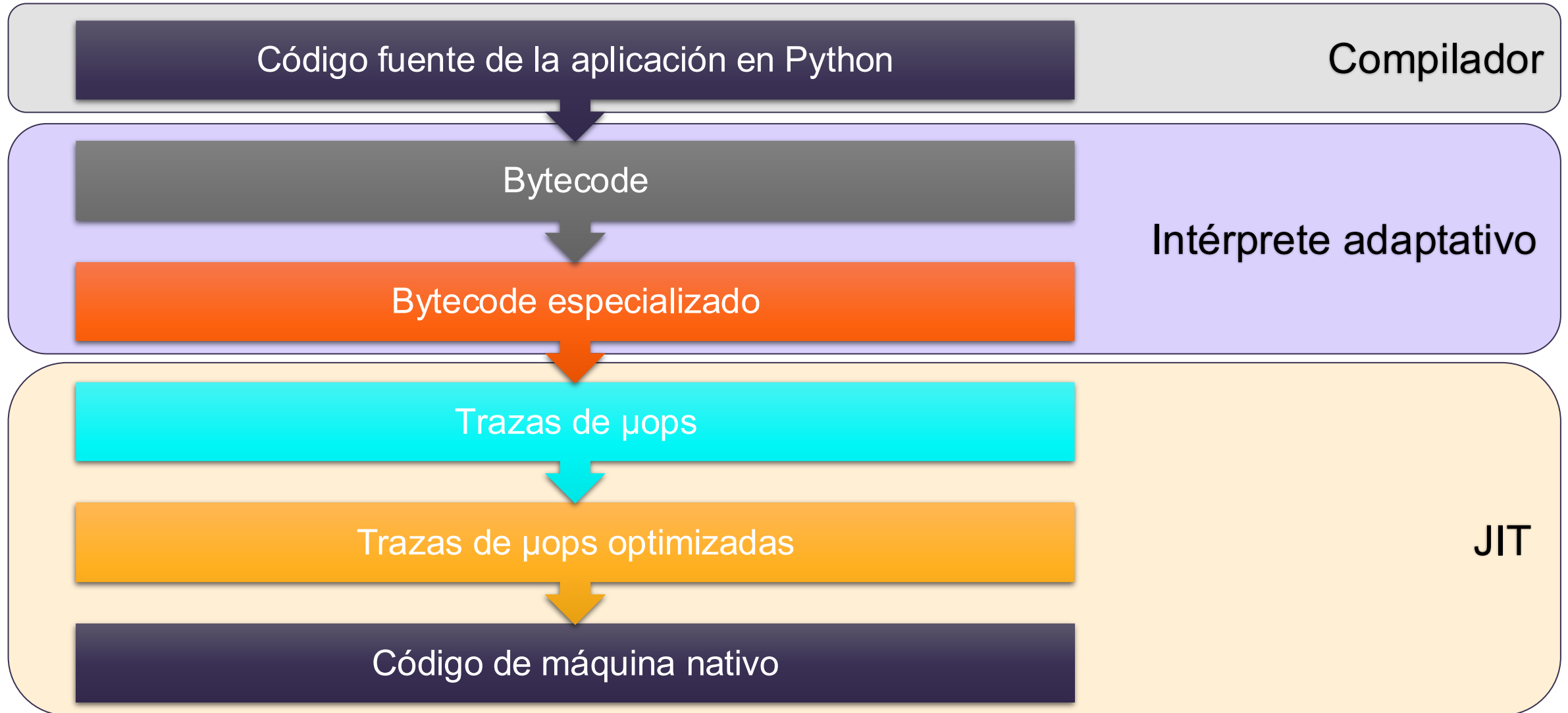
De la ejecución adaptativa a la compilada

Algunas rutas calientes pueden pasar de:



Cómo CPython ejecuta y optimiza tu código

No todo código atraviesa todas las etapas



Ejemplo: bucle crítico

Código fuente

```
def b():  
    total = 0  
    for i in range(10000):  
        total += i
```

Bytecode de CPython

```
1  RESUME_CHECK 0 0  
2  LOAD_SMALL_INT 0  
   STORE_FAST 0 (total)  
3  LOAD_GLOBAL 1 (range + NULL)  
   LOAD_CONST 1 (10000)  
   CALL 1  
   GET_ITER 0  
L1: FOR_ITER_RANGE 11 (to L2)  
   STORE_FAST 1 (i)  
4  LOAD_FAST_BORROW_LOAD_FAST_BORROW 1 (total, i)  
   BINARY_OP_ADD_INT 13 (+=)  
   STORE_FAST 0 (total)  
   JUMP_BACKWARD_NO_13 (to L1)  
3 L2: END_FOR  
   POP_ITER  
   LOAD_CONST 2 (None)  
   RETURN_VALUE
```

Ejemplo: bucle crítico

Código fuente

```
def b():  
    total = 0  
    for i in range(10000):  
        total += i
```

Bytecode especializado de CPython

```
1  RESUME_CHECK      0  
2  LOAD_SMALL_INT    0  
   STORE_FAST        0 (total)  
3  LOAD_GLOBAL        1 (range + NULL)  
   LOAD_CONST         1 (10000)  
   CALL               1  
   GET_ITER           0  
L1: FOR_ITER_RANGE    11 (to L2)  
   STORE_FAST         1 (i)  
4  LOAD_FAST_BORROW_LOAD_FAST_BORROW 1 (total, i)  
   BINARY_OP_ADD_INT  13 (+=  
   STORE_FAST         0 (total)  
   JUMP_BACKWARD_NO_JIT 13 (to L1)  
3 L2: END_FOR  
   POP_ITER  
   LOAD_CONST         2 (None)  
   RETURN_VALUE
```

Ejemplo: bucle crítico

Trazas de μops

FOR_ITER_RANGE	BINARY_OP_ADD_INT
_CHECK_VALIDITY	_CHECK_VALIDITY
_SET_IP	_SET_IP
_ITER_CHECK_RANGE	_GUARD_TOS_INT
_GUARD_NOT_EXHAUSTED_RANGE	_GUARD_NOS_INT
_ITER_NEXT_RANGE	_BINARY_OP_ADD_INT
STORE_FAST	_POP_TOP_INT
_CHECK_VALIDITY	_POP_TOP_INT
_SET_IP	STORE_FAST
_SWAP_FAST	_CHECK_VALIDITY
_POP_TOP	_SET_IP
LOAD_FAST_BORROW_LOAD_FAST_BORROW	_SWAP_FAST
_CHECK_VALIDITY	_POP_TOP
_SET_IP	_JUMP_TO_TOP
_LOAD_FAST_BORROW	JUMP_BACKWARD
_LOAD_FAST_BORROW	_CHECK_VALIDITY
	_SET_IP
	_CHECK_PERIODIC

Trazas de μops optimizadas

_CHECK_VALIDITY_r00
_ITER_CHECK_RANGE_r02
_GUARD_NOT_EXHAUSTED_RANGE_r22
_ITER_NEXT_RANGE_r23
_SET_IP_r33
_SWAP_FAST_1_r33
_SPILL_OR_RELOAD_r31
_POP_TOP_r10
_CHECK_VALIDITY_r00
_LOAD_FAST_BORROW_0_r01
_LOAD_FAST_BORROW_1_r12
_GUARD_TOS_OVERFLOWED_r22
_GUARD_NOS_INT_r22
_BINARY_OP_ADD_INT_r23
_POP_TOP_NOP_r32
_POP_TOP_NOP_r21
_SWAP_FAST_0_r11
_POP_TOP_INT_r10
_JUMP_TO_TOP_r00

Código de máquina

a8 82 5f f8 e9 03 28 2a 3f 05 40 f2 20 01 00 54 08 f9 7f 92 09 00 00 d0 08
05 40 f9 29 ed 46 f9 1f 01 09 eb 40 00 00 54 66 06 00 14 0d 00 00 14 fd 7b
bf a9 08 00 00 d0 00 00 00 d0 00 80 35 91 08 f1 46 f9 01 00 00 d0 21 f4 35
91 03 00 00 d0 63 80 36 91 22 3f 80 52 fd 03 00 91 00 01 3f d6 fd 7b bf a9
a8 82 5f f8 fd 03 00 91 e9 03 28 2a 3f 05 40 f2 c0 01 00 54 08 f9 7f 92 0a
00 00 d0 09 05 40 f9 4a 75 47 f9 3f 01 0a eb 41 02 00 54 08 11 40 f9 1f 01
00 f1 6d 00 00 54 fd 7b c1 a8 18 00 00 14 fd 7b c1 a8 0c 07 00 14 08 00 00
d0 00 00 00 d0 00 d4 39 91 08 79 47 f9 01 00 00 d0 21 48 3a 91 03 00 00
d0 63 d4 3a 91 22 3f 80 52 00 01 3f d6 08 00 00 d0 00 00 00 d0 00 a0 37
91 08 79 47 f9 01 00 00 d0 21 20 38 91 03 00 00 d0 63 94 38 91 42 0f 80
52 00 01 3f d6 1f 20 03 d5 fd 7b bf a9 a8 82 5f f8 fd 03 00 91 e9 03 28 2a
3f 05 40 f2 e0 06 00 54 08 f9 7f 92 0a 00 00 f0 09 05 40 f9 4a 79 40 f9 3f
01 0a eb 61 07 00 54 09 11 40 f9 3f 01 00 f1 4d 08 00 91 62 0f 80 52 00 01
3f d6 08 00 00 f0 00 00 00 d0 00 ec 3d 91 08 85 40 f9 01 00 00 d0 21 60 3c
91 03 00 00 d0 63 ac 3c 91 e2 0f 80 52 00 01 3f d6 08 00 00 f0 00 00 00 d0
00 18 3e 91 08 85 40 f9 01 00 00 d0 21 60 3c 91 03 00 00 d0 63 ac 3c 91
42 11 80 52 00 01 3f d6 08 00 00 f0 00 00 00 f0 00 dc 00 91 08 85 40 f9 01
00 00 d0 21 e4 3e 91 03 00 00 f0 63 3c 00 91 62 3b 80 52 00 01 3f d6 08
00 00 f0 00 00 00 f0 00 48 01 91 08 85 40 f9 01 00 00 f0 21 d8 01 91 03 00
00 f0 63 70 02 91 a2 02 80 52 00 01 3f d6 08 00 00 f0 00 00 00 f0 00 48 03
91

Cambiamos Python

Veamos cómo es contribuir a CPython en la práctica

Tres formas de cambiar Python

Tres cambios diferentes, el mismo proceso de contribución



Biblioteca estándar



Intérprete



Lenguaje



Biblioteca estándar

La biblioteca estándar también es parte de CPython

- ¿Cómo agrego una función nueva?
- Agregamos una función nueva en Lib/glob.py

`glob.glob(..., recursive=True)` → `glob.rglob(...)`



Intérprete

Los tipos incorporados también pueden evolucionar

- ¿Cómo agrego un método a list?
- Cambiamos el archivo Objects/listobject.c

list.insert(0, value) → list.prepend(value)



Lenguaje

La sintaxis también forma parte del código fuente de CPython

- ¿Cómo cambio la sintaxis de Python?
- Cambiamos el archivo Grammar/python.gram

if	→	si
elif	→	sinosi
return	→	retornar

¿Y ahora qué?

Cómo dar los primeros pasos en CPython

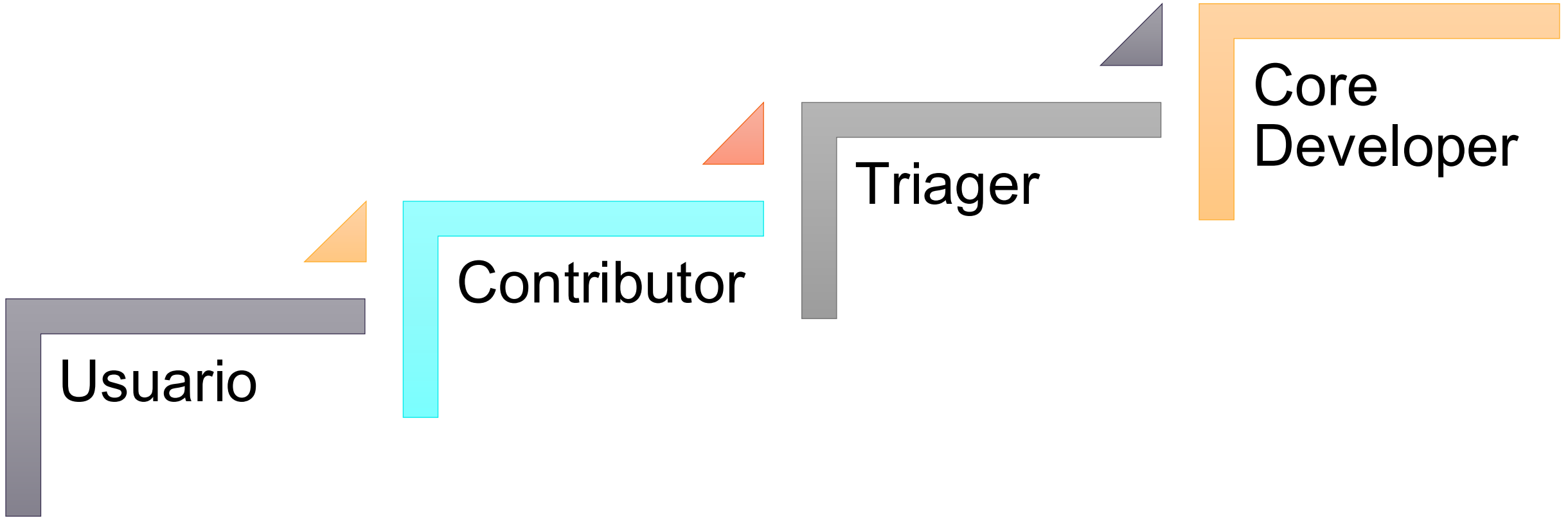
Hay muchas formas de contribuir

Todas son contribuciones valiosas para el proyecto



Un recorrido posible

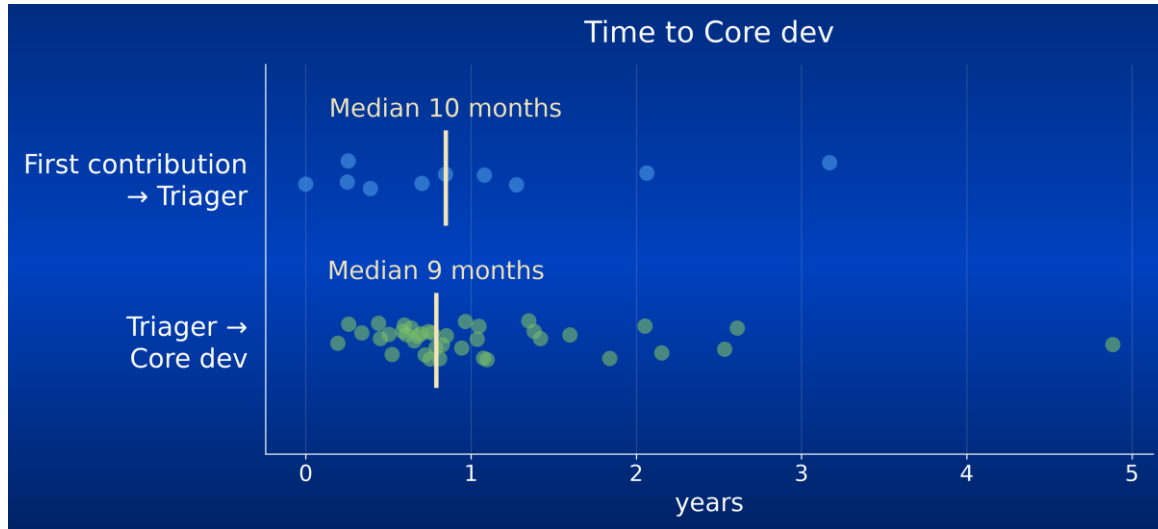
Más experiencia, más responsabilidad, más confianza



No existe un número mágico

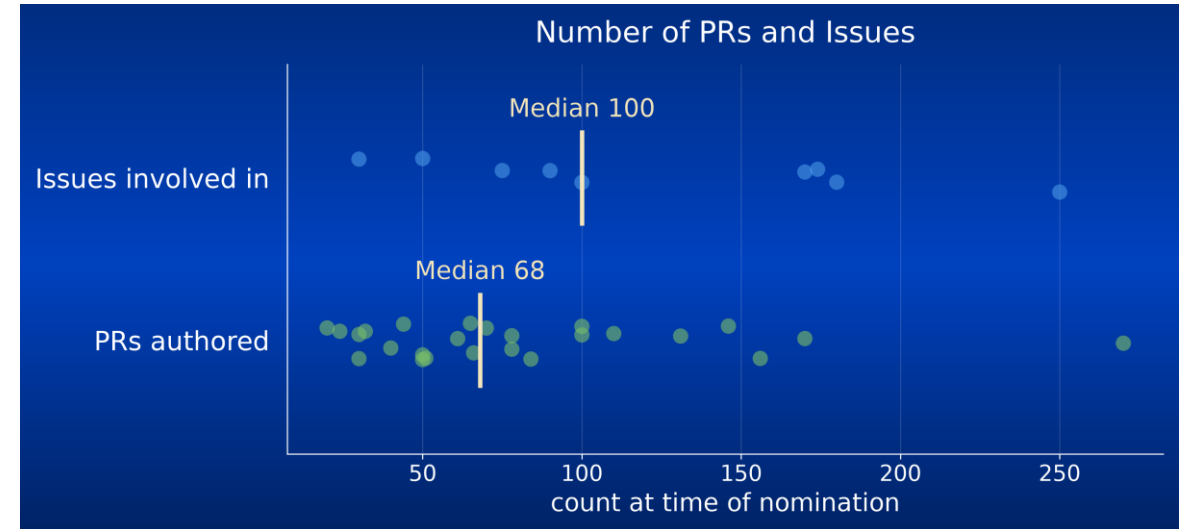
Cada recorrido es distinto, lo importante es la constancia

Tiempo



- Mediana: 10 meses
 - Primera contribución → Triager
- Mediana: 9 meses
 - Triager → Core Developer

Actividad



- Mediana: 100 issues en los que participó
- Mediana: 68 PRs creados

Fuente: Tomáš Roun, EuroPython 2026

Un posible primer mes

No hace falta empezar con cambios grandes



Semana 1



Semana 2



Semana 3



Semana 4



Compilar CPython
Correr tests

Reproducir un issue
Comentar

Enviar un cambio
pequeño
Escribir tests

Leer un PR y hacer una
pregunta útil

La IA puede ayudar, pero el aporte sigue siendo tuyo

La velocidad de generar código no reemplaza el criterio de ingeniería

Usala para



- ☐ 🔍 Entender código existente
- ☐ 🧪 Explorar tests y documentación
- ☐ 📖 Mejorar una explicación
- ☐ 💡 Aprender conceptos nuevos

Antes de enviar algo



- ☐ 🧠 Confirmá que resuelve un problema real
- ☐ ✅ Revisá cada cambio
- ☐ 🗣️ Entendelo y explicalo con tus palabras
- ☐ 🎯 Mantenelo pequeño, enfocado y probado

¿Por dónde empezar?

La curiosidad es el mejor punto de partida



Guía de desarrollo de
Python

<https://devguide.python.org>



Repositorio de
Python

<https://github.com/python/cpython>

Tutorial

https://github.com/cmaureir/cpython_tutorial



Foro oficial de Python

<https://discuss.python.org>



Conferencias

<https://wiki.python.org/moin/PythonConferences>

Lo que me hubiera gustado saber

La primera contribución no tiene que ser importante. Tiene que existir

Nadie entiende todo CPython

Todos empezamos con un primer aporte

El feedback es parte del aprendizaje

La comunidad necesita personas nuevas

Así se construye Python

Python evoluciona con miles de contribuciones hechas por personas de todo el mundo



El próximo aporte puede ser el de alguno de ustedes.

¿Preguntas?



diego@python.org



<https://www.diegor.it>



<https://github.com/diegorusso>